



Instituto Tecnológico de Aeronáutica
Divisão de Engenharia Aeronáutica e Aeroespacial

Laboratório 6

Processamento de Linguagem com RNN

Deep Learning

Autor:

João Vitor Coelho Guimarães Lima

Professores:

Prof. Marcos Ricardo Omena de Albuquerque Maximo

Prof. André Oliveira Françani

21 de junho de 2026

1 Introdução

O presente relatório descreve a implementação, o treinamento e a avaliação de **redes neurais recorrentes** aplicadas à **classificação de sentimento** (positivo ou negativo) de avaliações de filmes da base **IMDB**. O conjunto de dados é composto por **25.000** amostras de treino e **25.000** amostras de teste, balanceadas entre as duas classes. Todo o desenvolvimento foi realizado utilizando a *framework* **PyTorch**, em conformidade com as especificações descritas no enunciado do trabalho.

O objetivo central da atividade é comparar três tipos de **célula recorrente** no mesmo problema: a **RNN básica** (vanilla), a **LSTM** (*Long Short-Term Memory*) e a **GRU** (*Gated Recurrent Unit*). A comparação permite avaliar, de forma controlada, o impacto dos mecanismos de **portões** (*gates*) e de **estado de memória** sobre o desempenho em sequências de texto relativamente longas, nas quais a preservação de dependências de longo prazo é determinante para a qualidade da classificação.

2 Metodologia

A seguir descrevem-se o pré-processamento dos dados, a arquitetura comum aos três modelos, o procedimento de treinamento e a contagem de parâmetros de cada variante.

2.1 Pré-processamento

O vocabulário foi construído a partir das **10.000 palavras mais frequentes** do conjunto de treino. Foram reservados **três tokens especiais**: **PAD = 0** (preenchimento), **START = 1** (início de sequência) e **OOV = 2** (*out-of-vocabulary*, para palavras fora do vocabulário). Cada avaliação foi convertida em uma **sequência de inteiros**, na qual cada índice representa uma palavra do vocabulário. Em seguida, aplicou-se **padding/truncamento à esquerda** para que todas as sequências tivessem um comprimento fixo de **500 tokens**: avaliações mais curtas são preenchidas com o token **PAD** no início, enquanto avaliações mais longas têm seu início descartado.

2.2 Arquitetura

Os três modelos compartilham a mesma estrutura, implementada na classe `RecurrentModel`, diferindo apenas no tipo de célula recorrente empregada. A arquitetura é composta pelas seguintes camadas:

- **Camada de *Embedding*** (vocabulário = 10.000, dimensão de saída = 32, `padding_idx = 0`): mapeia cada índice inteiro para um vetor denso de 32 dimensões, aprendendo uma representação vetorial das palavras. O parâmetro `padding_idx = 0` garante que o token de *padding* não contribua para o aprendizado.
- **Camada recorrente com 32 unidades** (RNN, LSTM ou GRU, com `batch_first = True`): processa a sequência de *embeddings* ao longo do tempo, mantendo um estado interno que resume a informação acumulada. É nesta camada que os três modelos se distinguem.
- **Uso apenas da última saída temporal**: do conjunto de saídas produzidas pela camada recorrente, utiliza-se somente a saída do **último passo de tempo**, interpretada como um resumo de toda a sequência.

- **Camada Linear(32, 1) com ativação *sigmoide***: projeta o vetor de 32 dimensões em um único valor escalar e o comprime ao intervalo $[0, 1]$, fornecendo a probabilidade de a avaliação ser positiva (classificação binária).

2.3 Treinamento

O treinamento utilizou a função de perda **BCELoss** (entropia cruzada binária), adequada à classificação binária, e o otimizador **RMSprop** com taxa de aprendizado 10^{-3} . Cada modelo foi treinado por **10 épocas**, com **batch_size = 32** e divisão **treino/validação de 80/20**. Ao longo do treinamento, salvou-se o **melhor modelo** segundo a **menor *val_loss***, evitando reter um modelo já em sobreajuste.

2.4 Número de parâmetros

A Tabela 1 apresenta o número total de parâmetros de cada modelo. A diferença entre eles decorre do **número de conjuntos de pesos/portões** de cada célula recorrente: a **RNN básica** possui apenas **um** conjunto de pesos, a **GRU** possui **três portões** e a **LSTM** possui **quatro portões**. Como o restante da rede (*embedding* e camada linear) é idêntico, é justamente a célula recorrente que explica a variação observada.

Tabela 1: Número de parâmetros de cada modelo.

Modelo	Parâmetros
RNN básica	322145
GRU	326369
LSTM	328481

3 Resultados

O desempenho dos três modelos foi avaliado a partir das curvas de **perda** e **acurácia** de treino e validação ao longo das épocas, bem como das métricas finais obtidas no **conjunto de teste**.

As Figuras 1 e 2 apresentam, respectivamente, as curvas de perda e de acurácia da **RNN básica**.

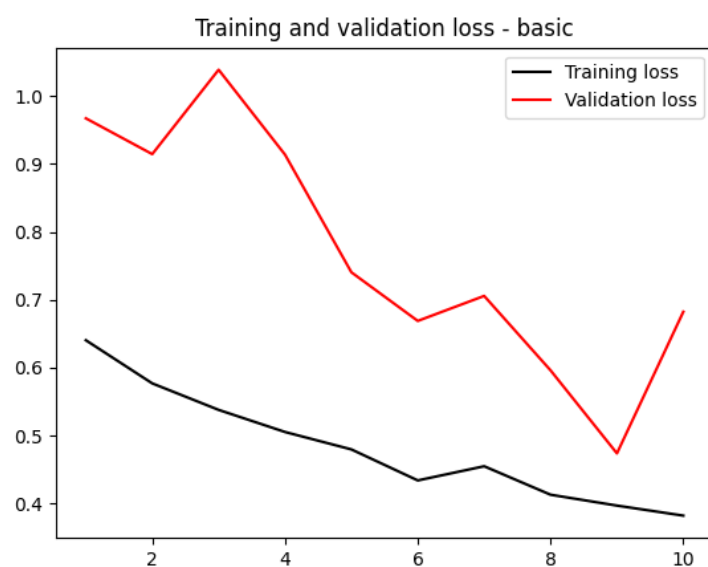


Figura 1: Perda de treino e validação da RNN básica ao longo das épocas.

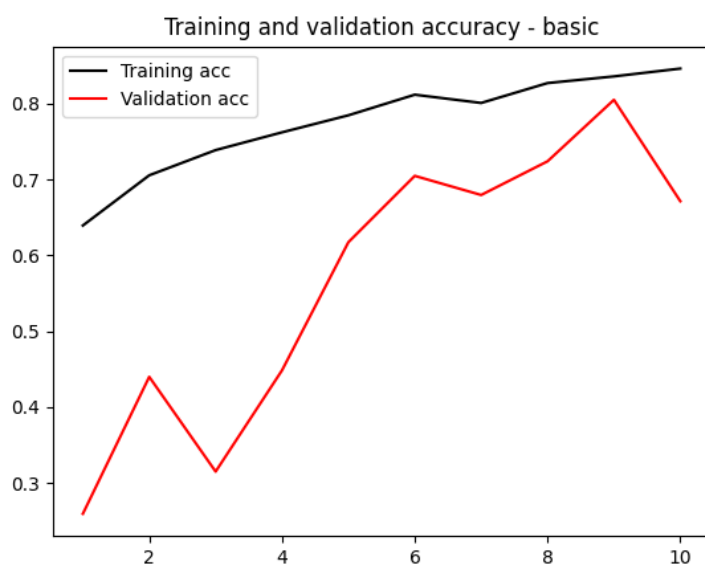


Figura 2: Acurácia de treino e validação da RNN básica ao longo das épocas.

As Figuras 3 e 4 exibem as curvas de perda e de acurácia da **LSTM**.

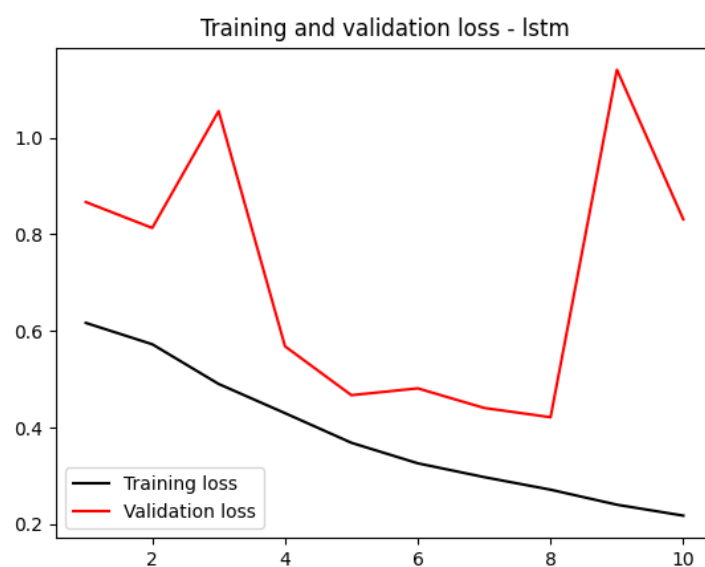


Figura 3: Perda de treino e validação da LSTM ao longo das épocas.

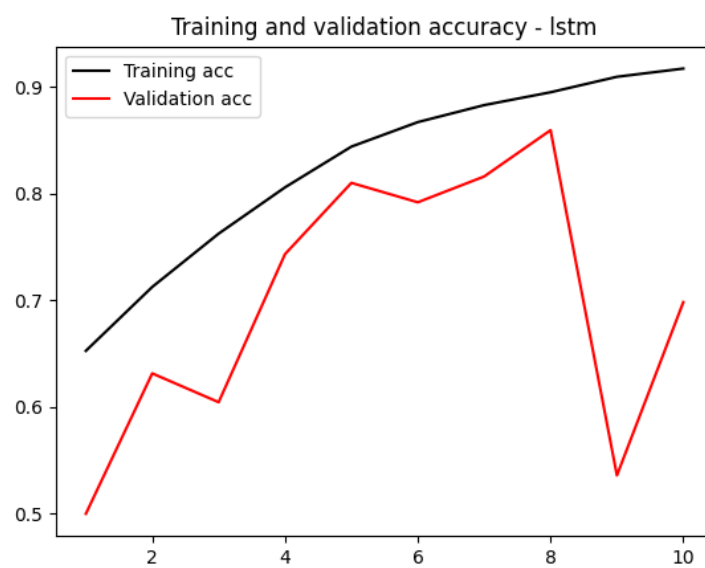


Figura 4: Acurácia de treino e validação da LSTM ao longo das épocas.

Por fim, as Figuras 5 e 6 mostram as curvas de perda e de acurácia da **GRU**.



Figura 5: Perda de treino e validação da GRU ao longo das épocas.

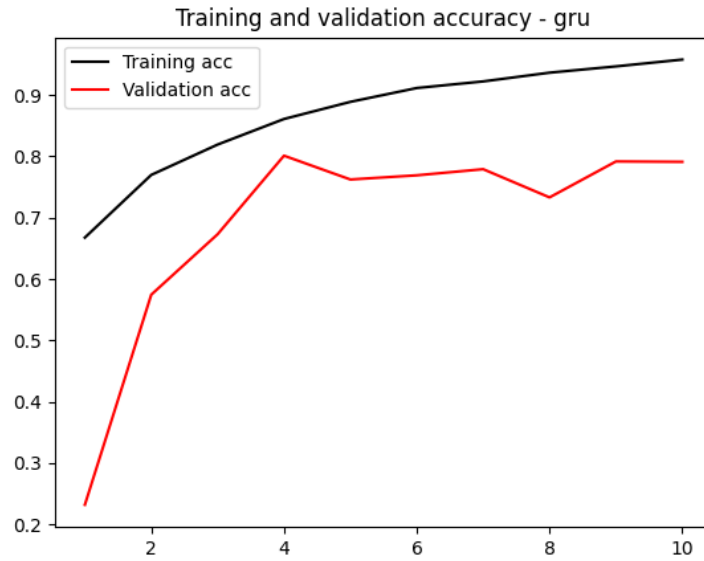


Figura 6: Acurácia de treino e validação da GRU ao longo das épocas.

Observa-se que a **acurácia de treino** cresce de forma consistente nos três modelos, atingindo, na **10^a época**, aproximadamente **84,6%** para a RNN básica, **91,7%** para a LSTM e **95,8%** para a GRU. Em contrapartida, as **curvas de validação** apresentam comportamento **ruidoso**, com **picos esporádicos de *val_loss*** — comportamento esperado e mencionado no enunciado do laboratório, decorrente da própria natureza do treinamento de redes recorrentes neste problema.

A Tabela 2 resume as métricas finais obtidas no **conjunto de teste** para os três modelos.

Tabela 2: Resultados no conjunto de teste.

Modelo	Loss (teste)	Acurácia (teste)
RNN básica	0,4959	78,58%
LSTM	0,3816	83,46%
GRU	0,3705	85,23%

3.1 Discussão

A **LSTM** (83,46%) e a **GRU** (85,23%) **superaram a marca de 80%** de acurácia no teste, atendendo ao critério de sucesso estabelecido no enunciado. Já a **RNN básica** ficou abaixo desse patamar, com **78,58%**.

Esse resultado **condiz com a teoria**. A **RNN básica** (vanilla) sofre com o **problema do *vanishing gradient*** em sequências longas — como as de até **500 tokens** usadas aqui —, o que dificulta ”lembrar” informações do **início da avaliação** ao chegar ao seu final. Em contraste, a **LSTM** e a **GRU**, graças aos seus **portões (*gates*)** e ao **estado de memória**, conseguem **preservar dependências de longo prazo**, resultando em desempenho superior.

Neste experimento, a **GRU** apresentou desempenho **ligeiramente superior** ao da LSTM. Esse comportamento é **coerente** com o fato de a GRU frequentemente **igualar ou superar** a LSTM em **bases menores** e com **menos parâmetros**. Ressalta-se, contudo, que a **diferença é pequena** e que as curvas de **validação são ruidosas**, de modo que a **ordem exata** entre LSTM e GRU pode **variar entre diferentes execuções**.

4 Conclusão

Os três modelos recorrentes foram **implementados e treinados com sucesso** na tarefa de classificação de sentimento da base IMDB. As células com **portões (LSTM e GRU)** **superaram a RNN básica**, confirmando a **vantagem dessas arquiteturas** no tratamento de **sequências longas**, em linha com a teoria sobre o *vanishing gradient*. Ambas **ultrapassaram a marca de 80%** de acurácia no conjunto de teste, atendendo ao critério de sucesso do laboratório, enquanto a RNN básica permaneceu abaixo desse limiar. Os resultados obtidos, portanto, **condizem com a teoria** apresentada e evidenciam a importância dos mecanismos de memória e de portões para a modelagem de dependências de longo prazo em sequências de texto.